Practical – 9

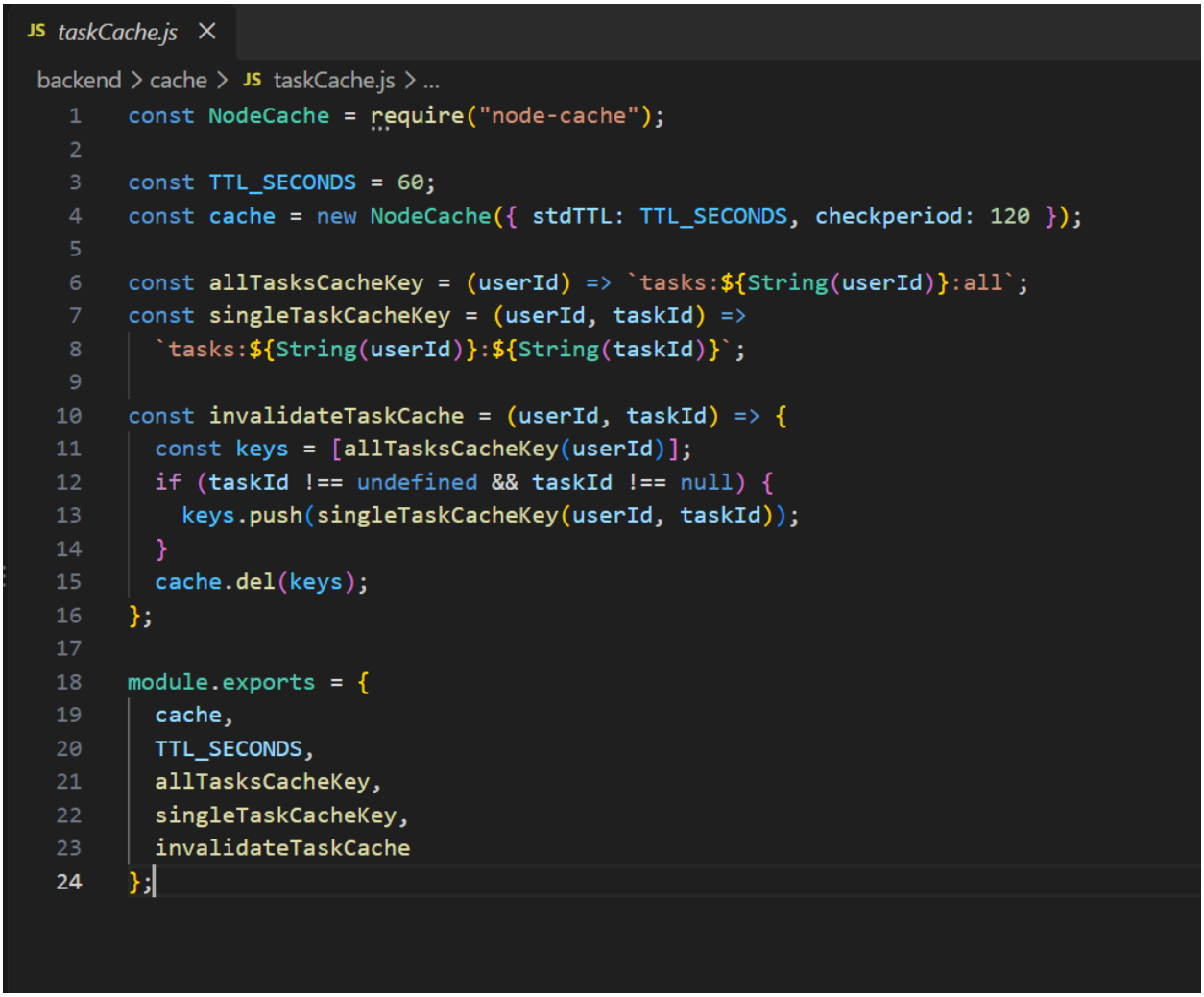
Aim:

To implement server-side caching and measure its impact on API response time.

Content:

After completing this practical, I will be able to improve the performance of a full-stack Task Management application using in-memory caching and query optimization. I will learn to implement caching using node-cache, reduce repeated database queries, invalidate cached data after create, update, and delete operations, and measure API response times using Postman. This practical will help me understand caching techniques, improve backend performance, and develop efficient and responsive web applications

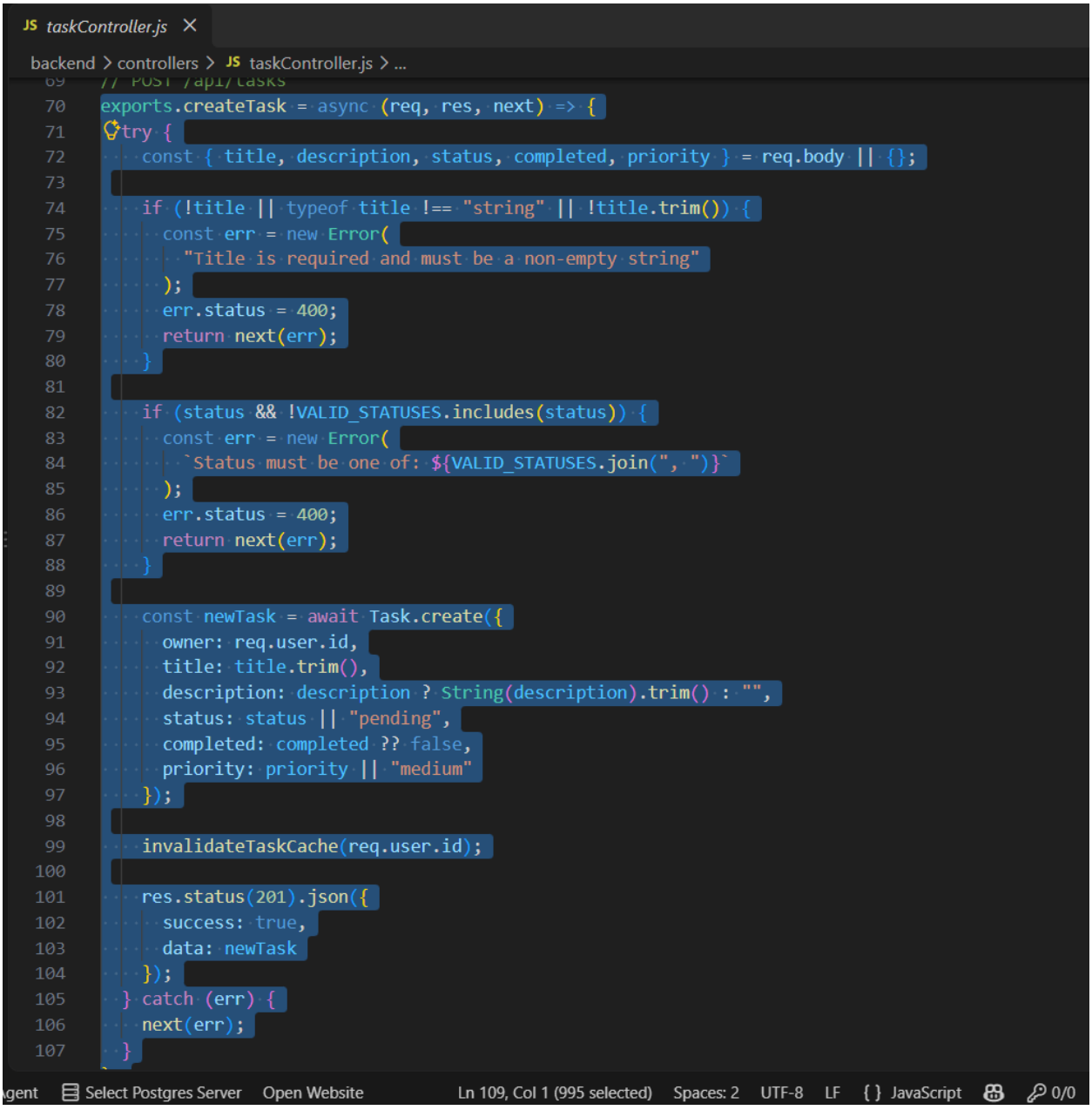
Screen Shot:

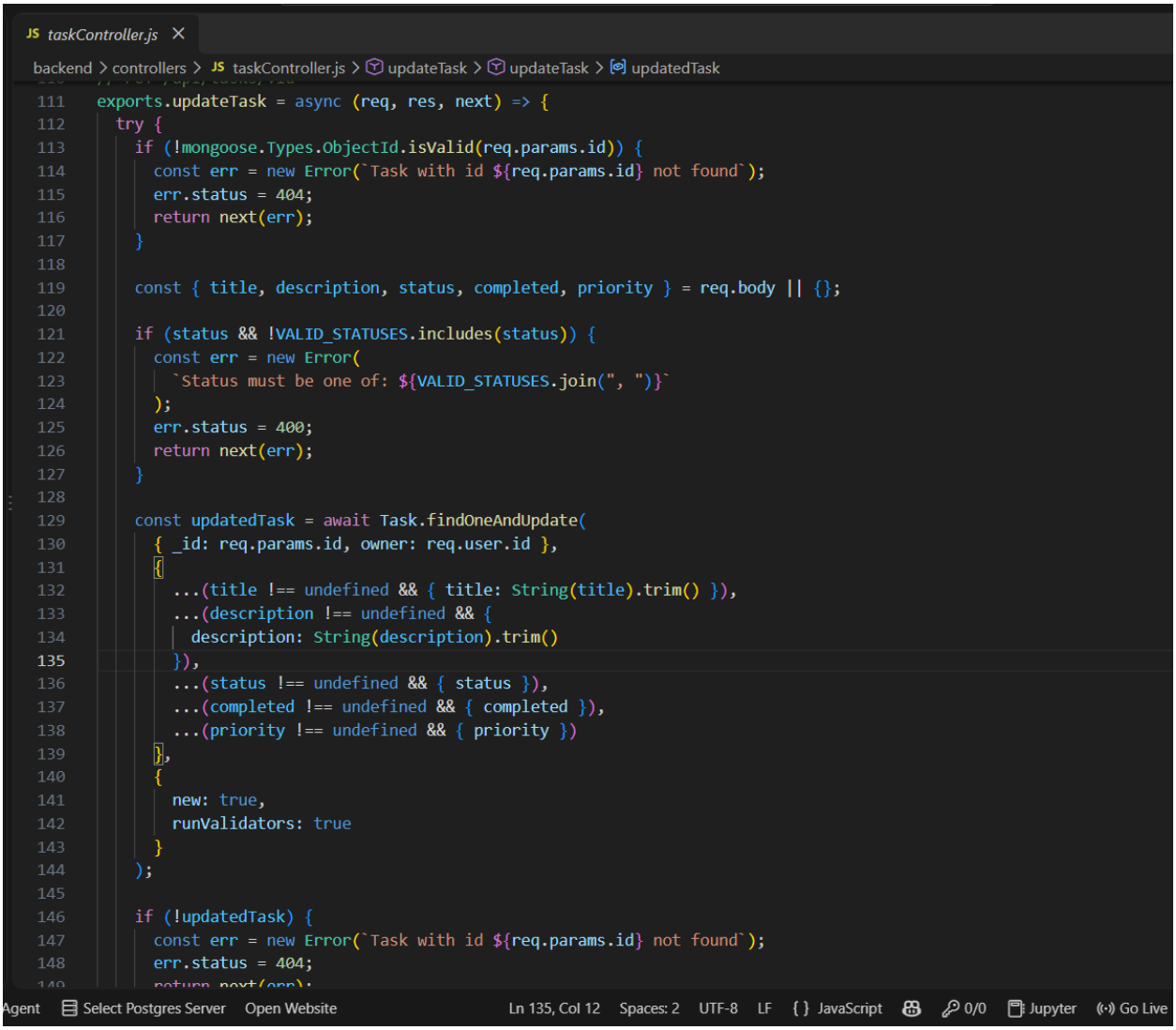


JS taskController.js

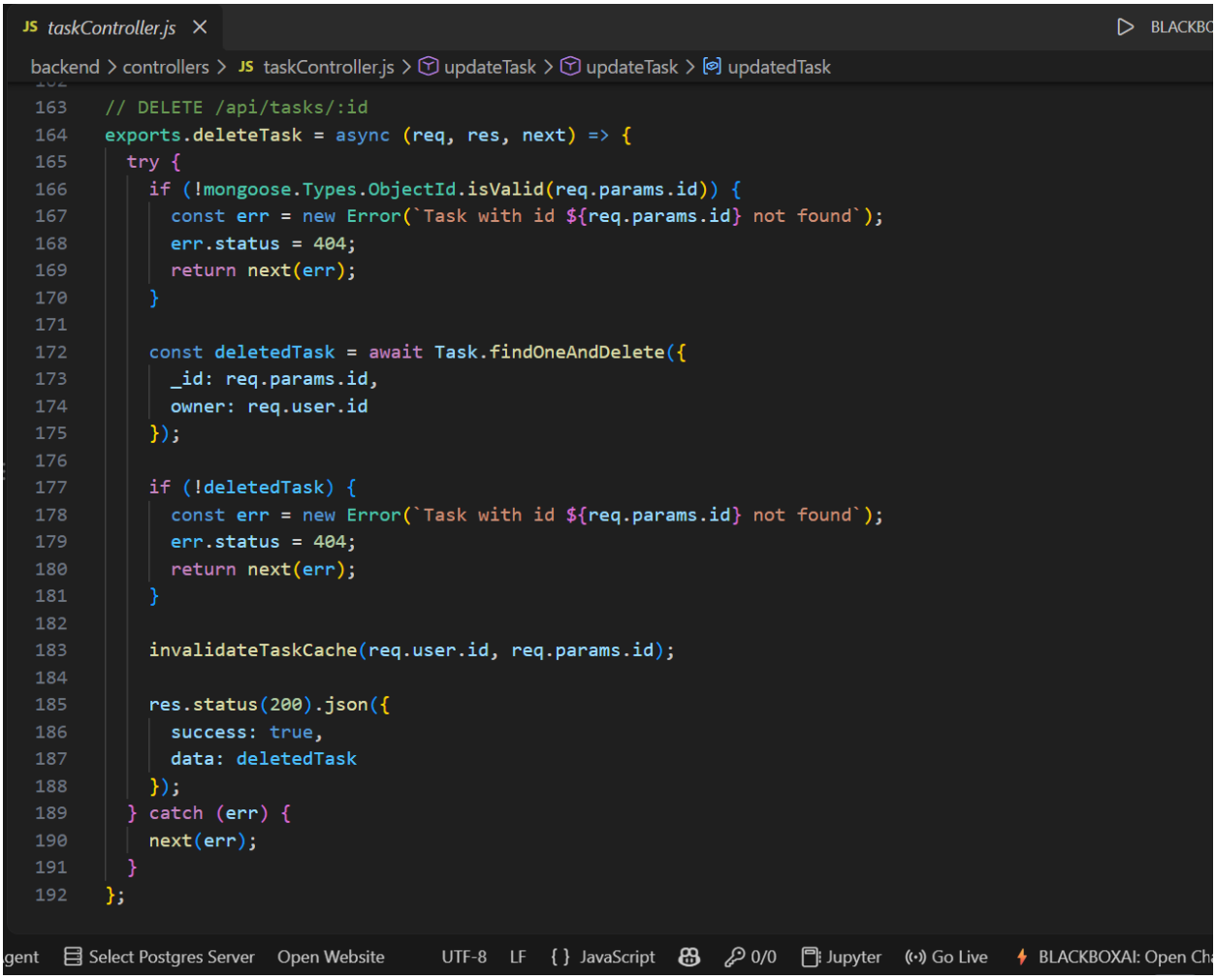
backend > controllers > JS taskController.js > getAllTasks > getAllTasks

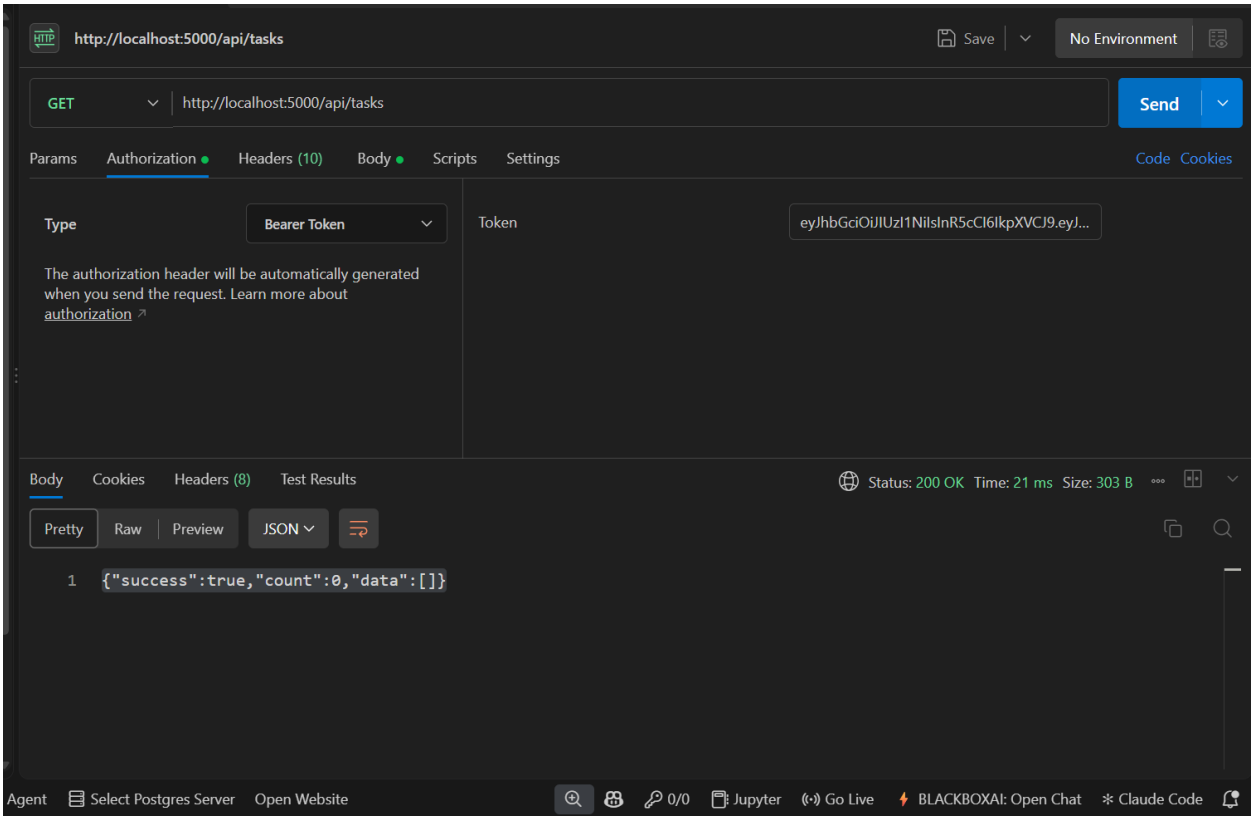
```
9
10  const VALID_STATUSES = ["pending", "in-progress", "done"];
11
12  // GET /api/tasks
13  exports.getAllTasks = async (req, res, next) => {
14    try {
15      const cacheKey = allTasksCacheKey(req.user.id);
16      const cachedResponse = cache.get(cacheKey);
17      if (cachedResponse !== undefined) {
18        return res.status(200).json(cachedResponse);
19      }
20
21      const tasks = await Task.find({ owner: req.user.id }).lean();
22      const response = {
23        success: true,
24        count: tasks.length,
25        data: tasks
26      };
27      cache.set(cacheKey, response);
28
29      res.status(200).json(response);
30    } catch (err) {
31      next(err);
32    }
33  };
34
```

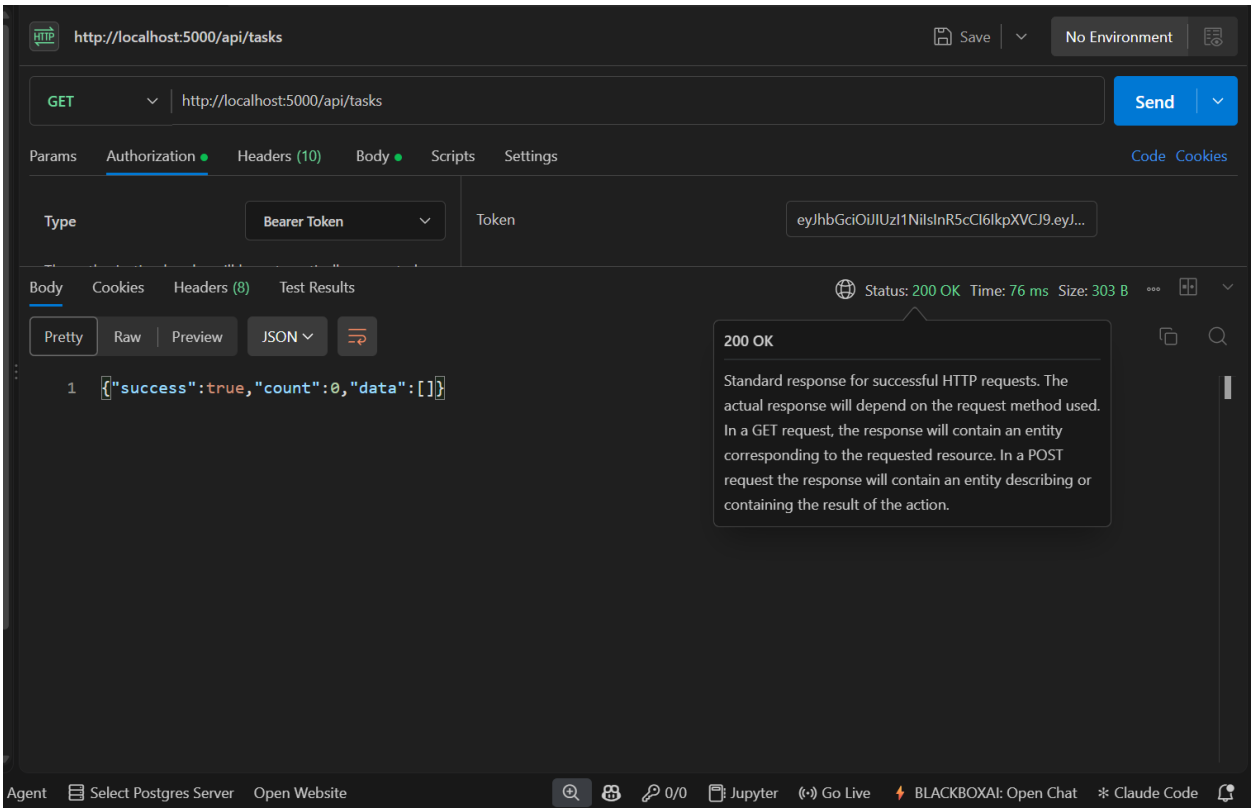

taskController.js
backend > controllers > taskController.js > ...
// POST /api/tasks
exports.createTask = async (req, res, next) => {
try {
const { title, description, status, completed, priority } = req.body || {};
if (!title || typeof title !== "string" || !title.trim()) {
const err = new Error(
"Title is required and must be a non-empty string"
);
err.status = 400;
return next(err);
}
if (status && !VALID_STATUSES.includes(status)) {
const err = new Error(
`Status must be one of: ${VALID_STATUSES.join(", ")}`
);
err.status = 400;
return next(err);
}
const newTask = await Task.create({
owner: req.user.id,
title: title.trim(),
description: description ? String(description).trim() : "",
status: status || "pending",
completed: completed ?? false,
priority: priority || "medium"
});
invalidateTaskCache(req.user.id);
res.status(201).json({
success: true,
data: newTask
});
} catch (err) {
next(err);
}
Select Postgres Server
Open Website
Ln 109, Col 1 (995 selected)
Spaces: 2
UTF-8
LF
{ } JavaScript
0/0

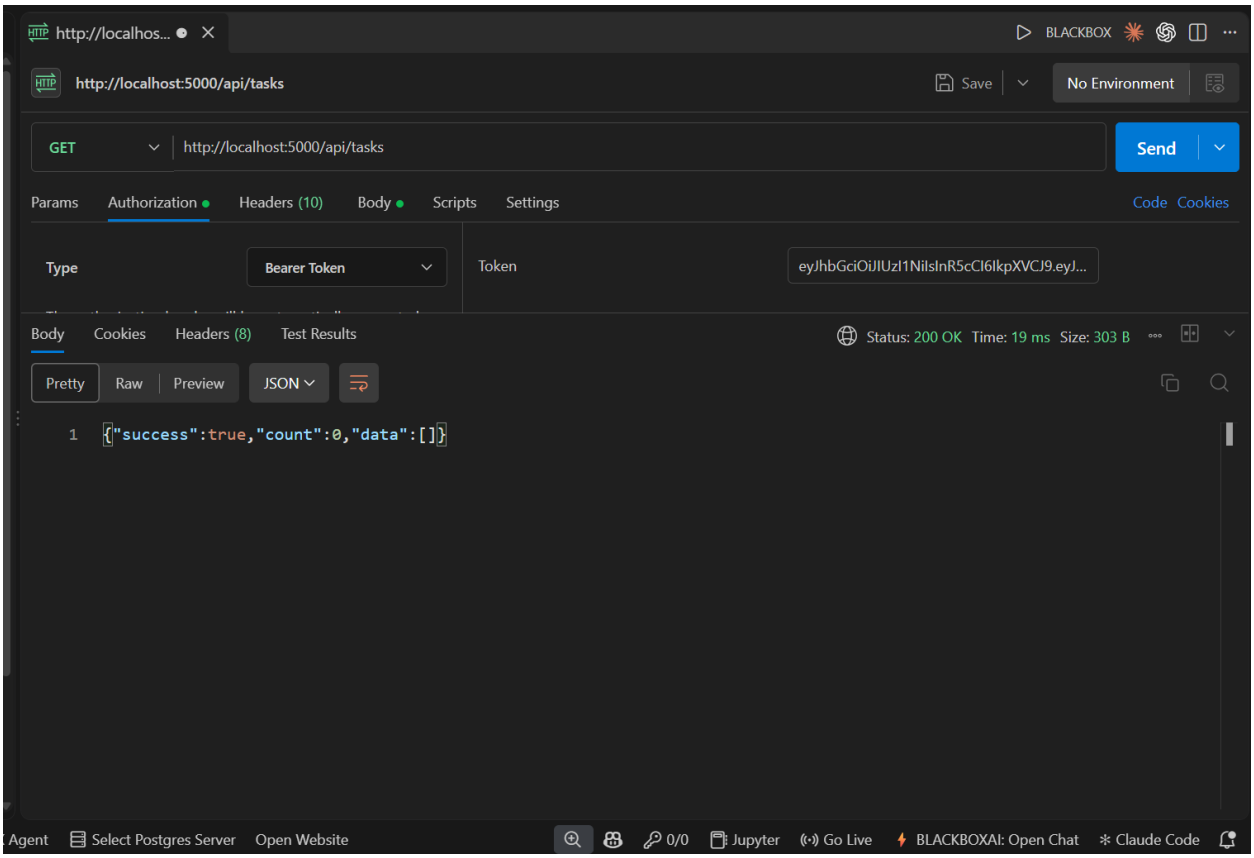


```
taskController.js
backend > controllers > taskController.js > updateTask > updateTask > updatedTask
111  exports.updateTask = async (req, res, next) => {
112    try {
113      if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
114        const err = new Error(`Task with id ${req.params.id} not found`);
115        err.status = 404;
116        return next(err);
117      }
118
119      const { title, description, status, completed, priority } = req.body || {};
120
121      if (status && !VALID_STATUSES.includes(status)) {
122        const err = new Error(
123          `Status must be one of: ${VALID_STATUSES.join(", ")}`
124        );
125        err.status = 400;
126        return next(err);
127      }
128
129      const updatedTask = await Task.findOneAndUpdate(
130        { _id: req.params.id, owner: req.user.id },
131        {
132          ...(title !== undefined && { title: String(title).trim() }),
133          ...(description !== undefined && {
134            description: String(description).trim()
135          }),
136          ...(status !== undefined && { status }),
137          ...(completed !== undefined && { completed }),
138          ...(priority !== undefined && { priority })
139        },
140        {
141          new: true,
142          runValidators: true
143        }
144      );
145
146      if (!updatedTask) {
147        const err = new Error(`Task with id ${req.params.id} not found`);
148        err.status = 404;
Agent  Select Postgres Server  Open Website  Ln 135, Col 12  Spaces: 2  UTF-8  LF  { } JavaScript  0/0  Jupyter  Go Live
```


taskController.js
backend > controllers > taskController.js > updateTask > updateTask > updatedTask
// DELETE /api/tasks/:id
exports.deleteTask = async (req, res, next) => {
try {
if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
const err = new Error(`Task with id ${req.params.id} not found`);
err.status = 404;
return next(err);
}
const deletedTask = await Task.findOneAndDelete({
_id: req.params.id,
owner: req.user.id
});
if (!deletedTask) {
const err = new Error(`Task with id ${req.params.id} not found`);
err.status = 404;
return next(err);
}
invalidateTaskCache(req.user.id, req.params.id);
res.status(200).json({
success: true,
data: deletedTask
});
} catch (err) {
next(err);
}
};
Select Postgres Server
Open Website
UTF-8
LF
JavaScript
0/0
Jupyter
Go Live


http://localhost:5000/api/tasks
Save
No Environment
GET
http://localhost:5000/api/tasks
Send
Params
Authorization
Headers (10)
Body
Scripts
Settings
Code
Cookies
Type
Bearer Token
Token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...
The authorization header will be automatically generated when you send the request. Learn more about authorization
Body
Cookies
Headers (8)
Test Results
Status: 200 OK
Time: 21 ms
Size: 303 B
Pretty
Raw
Preview
JSON
1 {"success":true,"count":0,"data":[]}
Agent
Select Postgres Server
Open Website
0/0
Jupyter
Go Live
BLACKBOXAI: Open Chat
Claude Code


http://localhost:5000/api/tasks
Save
No Environment
GET
http://localhost:5000/api/tasks
Send
Params
Authorization
Headers (10)
Body
Scripts
Settings
Code
Cookies
Type
Bearer Token
Token
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ...
Body
Cookies
Headers (8)
Test Results
Status: 200 OK
Time: 76 ms
Size: 303 B
Pretty
Raw
Preview
JSON
1 {"success":true,"count":0,"data":[]}
200 OK
Standard response for successful HTTP requests. The actual response will depend on the request method used. In a GET request, the response will contain an entity corresponding to the requested resource. In a POST request the response will contain an entity describing or containing the result of the action.
Agent
Select Postgres Server
Open Website
0/0
Jupyter
Go Live
BLACKBOXAI: Open Chat
Claude Code

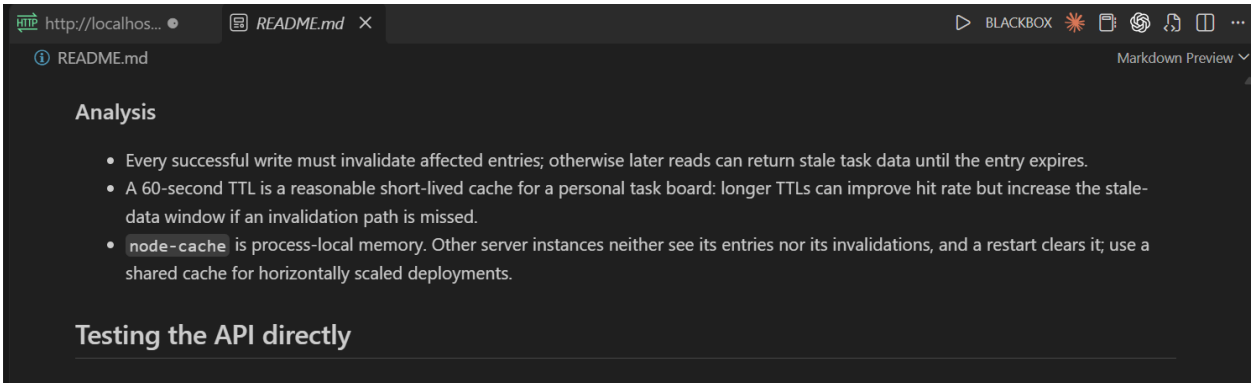


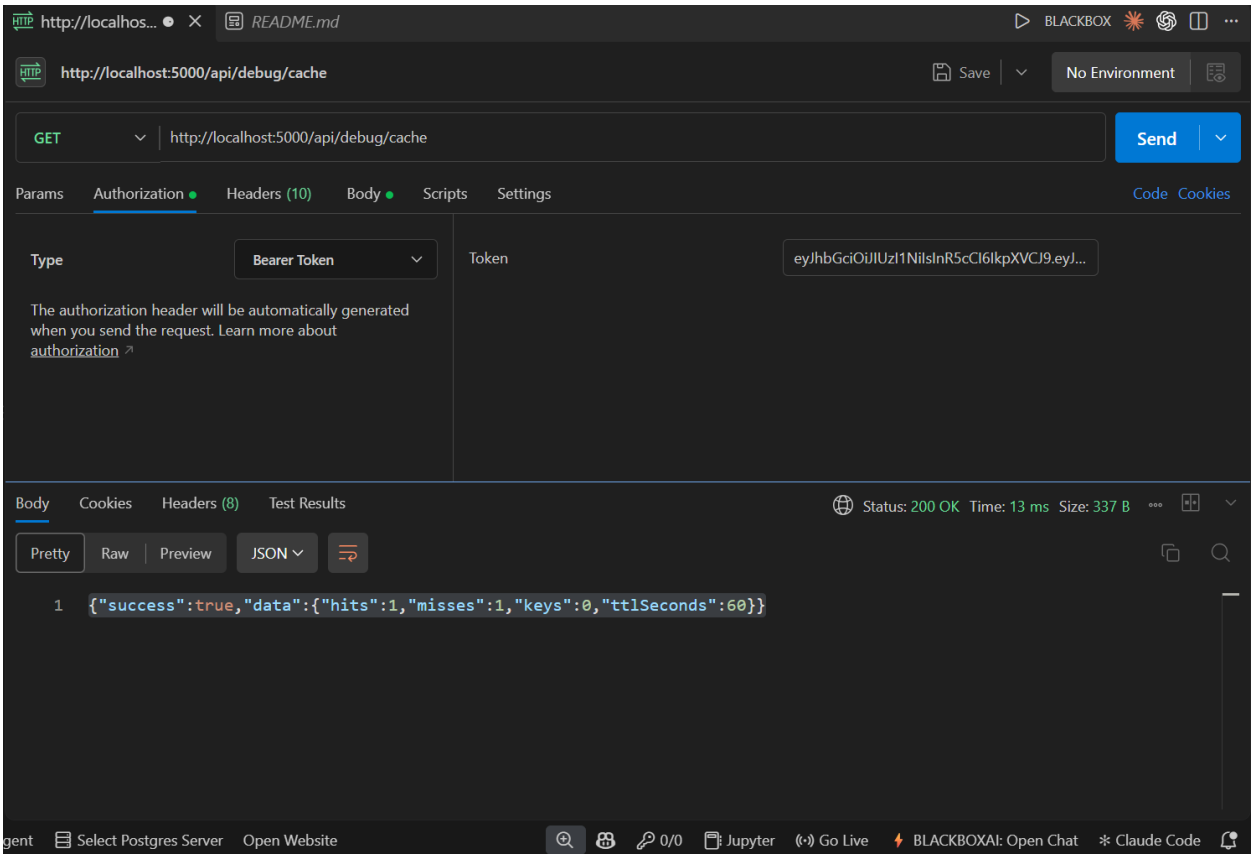
Response-time comparison

Three uncached samples were recorded before the cache change. For the cached set, one request warmed the cache before timing three subsequent requests. Both sets used the same authenticated `GET /api/tasks` endpoint, one owner with one task, local Node `fetch`, and the same machine/API process.

Sample	Uncached	Cached
1	71.43 ms	9.53 ms
2	10.78 ms	15.65 ms
3	12.88 ms	5.53 ms
Mean	**31.70 ms**	**10.24 ms**

In this small local run, cached mean response time was about 67.7% lower. The first uncached request was notably slower than the other two, and these results are not a production benchmark: the dataset is tiny and local timings vary. For a lab submission, repeat the readings in Postman or Thunder Client with a larger dataset and record the environment and cache-warm procedure.





Conclusion:

This practical demonstrates the implementation of in-memory caching and query optimization in a full-stack Task Management application using Node.js, Express.js, MongoDB, and Mongoose. It includes caching GET requests using node-cache, reducing repeated database queries, and invalidating cached data after create, update, and delete operations. It also covers API response time measurement, performance comparison using Postman, and testing cache functionality to improve backend performance and efficiency.